# Phase 1 Report — NoSQL ETL Framework

# Phase 1 Report

## Multi-Pipeline ETL & Reporting Framework for Web Server Log Analytics

**Course:** DAS 839 — NoSQL Systems, End-Semester Project **Phase:** 1 (architecture + working prototype) **Pipeline implemented in Phase 1:** MongoDB **Pipelines deferred to Phase 2:** Apache Pig, Apache Hive, Hadoop MapReduce

---

## Github Link:

https://github.com/jayantt23/multi-pipeline-log-analytics

## Executive Summary

We have built a runnable prototype of the multi-pipeline ETL framework with the **MongoDB** backend implemented end-to-end and the other three backends stubbed against a shared `Pipeline` interface. On the full NASA HTTP access logs (Jul95 + Aug95 = **3,461,613 lines**), the system parses, batches (`batch_size = 100,000` → 35 batches), executes the three mandated queries, and loads the results into PostgreSQL in **~114 seconds**. The framework's design choices (native ingestion + native batching + an engine-agnostic loader) are deliberate so that Phase 2 backends can plug in without altering parsing semantics, schema, or reporting code.

This report addresses the six discussion areas the project statement asks for in the Phase 1 review:

1. Overall design / architecture
2. Parsing strategy
3. ETL workflow
4. Batching approach
5. Relational reporting schema
6. Plan for ensuring equivalence across pipelines

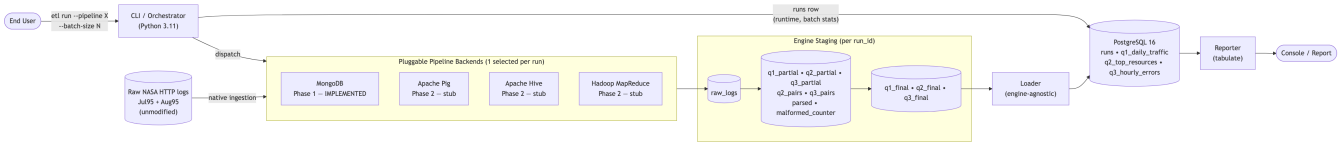
It closes with the working-prototype evidence from the demo run.

---

# 1. Overall Design

The system is a thin Python orchestrator around four pluggable execution backends. The orchestrator is responsible for I/O coordination, timing, and writing the run-metadata row to PostgreSQL; **all data processing happens inside the chosen backend** (MongoDB in Phase 1). Final results land in four PostgreSQL tables that are written by a backend-agnostic loader.

## 1.1 Component diagram




## 1.2 Module layout

```
NoSQL_Project_ETL_Analysis_Pipeline/
├── PARSING_SPEC.md              ← frozen rules; every backend must comply
├── PHASE1_REPORT.md             ← this document
├── docker-compose.yml           ← mongo:7 + postgres:16 (alternative to local installs)
├── pyproject.toml
├── sql/schema.sql               ← runs / q1 / q2 / q3 DDL
├── src/
│   ├── cli.py                   ← argparse: `run`, `report`
│   ├── orchestrator.py          ← dispatch + timer + runs row + cleanup
│   ├── loader.py                ← engine-agnostic q*_final → Postgres
│   ├── reporting/report.py      ← tabulate output
│   └── pipelines/
│       ├── base.py              ← Pipeline ABC + RunStats dataclass
│       ├── mongodb/             ← Phase 1: full implementation
│       │   ├── etl_pipeline.py  ← wires aggregations + ingestion
│       │   └── aggregations.py  ← pure-data agg builders
│       └── pig/, hive/, mr/     ← Phase 2: stubs raising NotImplementedError
└── tests/                       ← parsing + batch-id + e2e (13 passing)
```

The `Pipeline` ABC (`src/pipelines/base.py`) defines the five lifecycle methods every backend must implement (`ingest`, `aggregate`, `final_merge`, `collect_run_stats`, `cleanup`). The orchestrator and loader only ever talk to this interface, which is what makes the framework genuinely engine-pluggable.

## 1.3 Decisions & rationale

| Decision | Choice | Reason |
|---|---|---|
| Orchestrator language | Python 3.11 | Best `argparse` / `subprocess` story, native pymongo and psycopg2-binary |
| Result store | PostgreSQL 16 | Project allows MySQL or Postgres; chose Postgres for `NUMERIC(20,6)` and `TIMESTAMPTZ` |
| Batching strategy | **Native, in-engine, logical** (computed by Mongo, not pre-sliced by Python) | Compliance with the project's "core data processing must genuinely happen in the chosen execution technology" rule |

| Decision | Choice | Reason |
|---|---|---|
| `batch_id` formula | `floor((ordinal - 1) / N) + 1` | Single integer formula, identical across all four engines |
| Distinct counts | **Exact**, via staged distinct-pair collections | Rejects HLL / probabilistic drift across engines |
| Phase 1 backend | MongoDB | Easiest path to a fully-runnable pipeline; aggregation framework natively supports regex parsing + windowed ranking |

---

## 2. Parsing Strategy

Parsing is **fully specified in `PARSING_SPEC.md`** and frozen for the duration of the project. Every backend (Phase 1 Mongo and Phase 2 Pig / Hive / MR) must implement these rules byte-for-byte. This is what makes cross-engine equivalence achievable.

### 2.1 Master regex (POSIX-compatible)

```
^(\S+) \S+ \S+ \[([^\]]+)\] "([^"]*)" (\d{3}|-) (\d+|-)$
```

Capture groups: `host`, `timestamp_raw`, `request_raw`, `status_raw`, `bytes_raw`. The same regex string is used as `$regexFind` input in MongoDB, will be passed to `regexp_extract` in Hive, `REGEX_EXTRACT_ALL` in Pig, and `Pattern.compile` in MapReduce.

### 2.2 Field-level rules

| Field | Source | Rule |
|---|---|---|
| `host` | capture 1 | first whitespace-delimited token |
| `timestamp_raw` | capture 2 | parsed via `%d/%b/%Y:%H:%M:%S %z` |
| `log_date`, `log_hour` | derived | extracted in the **timestamp's own offset** — no UTC conversion (so engines with different default TZ behaviour produce identical results) |
| `http_method` / `resource_path` / `protocol_version` | capture 3 split on whitespace | tokens 1, 2, 3 — missing tokens become `NULL`, **not malformed** |
| `bytes_transferred` | capture 5 | `"-"` → `0`; numeric otherwise |
| `status_code` | capture 4 | `"-"` → record is **malformed** |

### 2.3 Malformed records

A record is malformed iff (a) the master regex fails to match, **or** (b) `status_raw == "-"`. Malformed records are **counted** (`malformed_count` in `runs`) and dropped from aggregation; they never reach Q1/Q2/Q3. They are **not** removed from the ordinal stream — that's important for batch-boundary stability (see §4).

## 2.4 Validation evidence

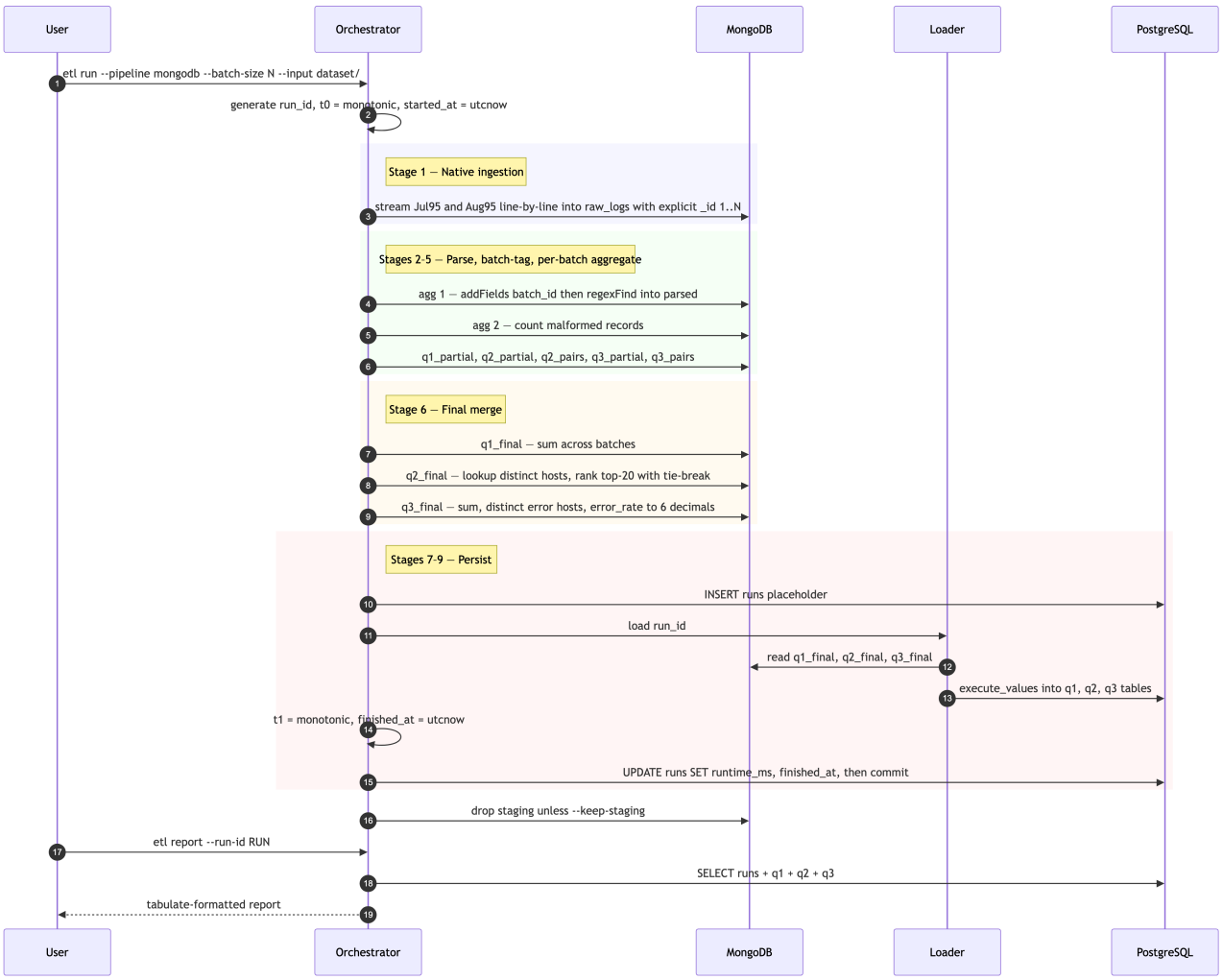
`tests/test_parsing.py` runs the exact `MASTER_REGEX` constant from `aggregations.py` against 11 hand-crafted lines (5 valid + 2 with `bytes='-'` + 2 with `status='-'` + 2 truly malformed). It asserts `malformed_count == 4` and verifies capture-group correctness, including that short-request lines are NOT malformed. On the real dataset, the run reported `malformed_count = 33` out of 3,461,613 lines (≈ 0.001%), consistent with the known small fraction of bad lines in the NASA traces.

---

# 3. ETL Workflow

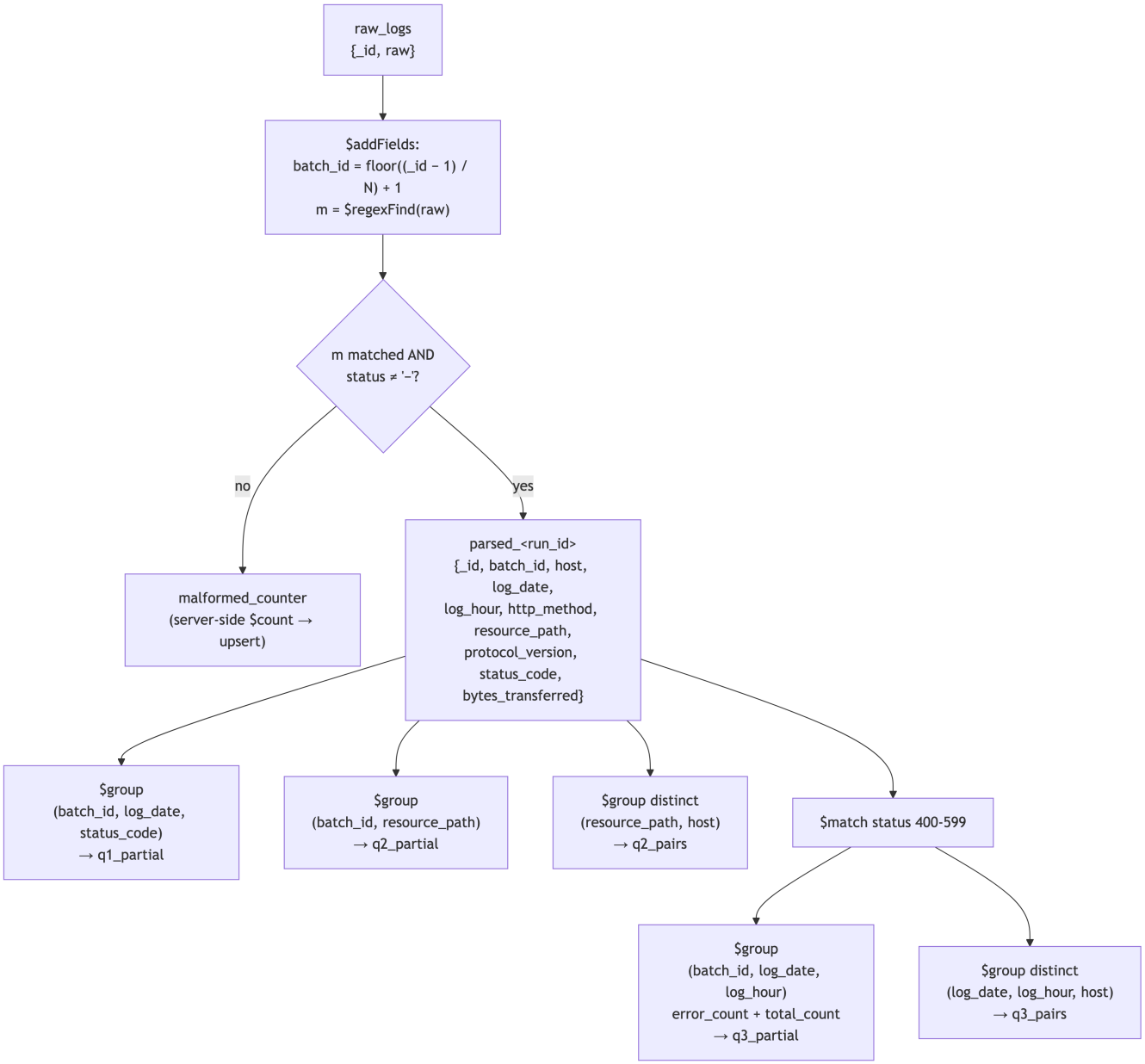
The workflow follows ten stages, of which stages 2–7 happen entirely inside MongoDB. Python only orchestrates and never touches a parsed field.

## 3.1 Sequence diagram




## 3.2 Per-batch dataflow inside MongoDB

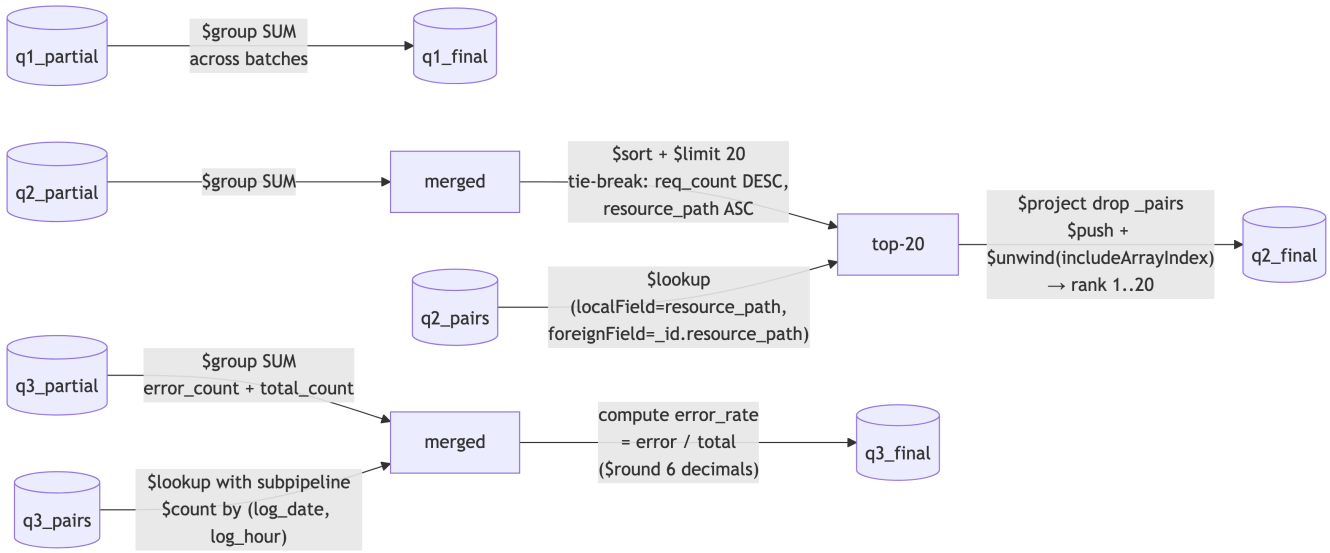
Inside the `aggregate()` step, after parse-and-tag has written `parsed_<run_id>`, five independent aggregations build the per-batch staging:




Two design notes worth highlighting:

- `q3_partial` carries **both** `error_request_count` (conditional sum: 1 iff 400 ≤ status ≤ 599) and `total_request_count` (unconditional sum). This avoids a second pass over `parsed` to compute the denominator of `error_rate`.
- `q2_pairs` and `q3_pairs` are **global** distinct collections (one row per unique pair / triple). They are not per-batch. The `$group` step itself dedupes across all input documents.

### 3.3 Final merge




Why `$push` + `$unwind` and not `$setWindowFields($documentNumber)` for Q2 ranking? `$documentNumber` requires a *single-element* `sortBy`, which can't express our two-key tie-break (`request_count DESC, resource_path ASC`). The `$sort + $limit(20)` step already establishes the canonical order; `$push` preserves that order, and `$unwind` with `includeArrayIndex` exposes the position as the rank. We also `$project: {_pairs: 0}` between `$lookup` and `$push` to discard the per-resource host arrays — without that, hot resources with 80k+ distinct hosts would push the in-memory accumulator past the 100 MB limit.

---

## 4. Batching Approach

### 4.1 Definition

The project statement defines batch size as **the number of input log records processed in one batch** (not bytes, not files, not aggregation chunks). Batch IDs start at 1, increase sequentially, and the last batch may be partial but is still counted.

### 4.2 Native, in-engine, logical batching

Two important properties of our implementation:

- **Native** — Python does not pre-slice the input. It only streams raw text into one collection (`raw_logs_<run_id>`) with explicit monotonically-increasing `_id` values 1..N. The batch boundaries are derived **inside MongoDB** via `$addFields` (see formula below).
- **Logical** — there is no physical separation: parsed records all sit in `parsed_<run_id>`; their batch is just a tag (`batch_id`). The same data can be re-batched at a different `N` simply by re-running the aggregation with a different parameter, no re-ingestion needed.

This satisfies the implementation constraint that "*core data processing must genuinely happen using the selected execution technology*". Pre-tokenising or pre-slicing in Python would be partial substitution, which the project statement disallows.

## 4.3 The formula

```
ordinal  = 1-indexed position of the record across the input files (lex-sorted)
batch_id = floor((ordinal - 1) / N) + 1
```

Worked example for `N = 10`, ordinals 1..25:

```
ordinal  : 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
batch_id : 1  1  1  1  1  1  1  1  1  1  2  2  2  2  2  2  2  2  2  2  3  3  3  3  3
```

Batches 1 and 2 are full (10 records each); batch 3 is partial (5 records). All three count toward `num_batches`. (Verified by `tests/test_batch_id.py`.)

## 4.4 Why ordinals are computed across all input lines

Including malformed lines. If we computed ordinals only over parsed records, then any change in the number of malformed lines (e.g. fixing a regex) would shift batch boundaries non-deterministically. By reserving an ordinal for every input line (malformed or not) and only excluding malformed records from aggregation later, batch boundaries are stable and reproducible.

## 4.5 Per-engine ordinal primitive (forward-looking)

| Engine | Ordinal source |
|---|---|
| MongoDB (Phase 1) | explicit `_id = 1..N` set during ingestion |
| Hive | `ROW_NUMBER() OVER (ORDER BY <stable_input_order>)` |
| Pig | `RANK <relation> BY ... DENSE` over the same stable order |
| MapReduce | line-offset emitted by the input format, then a single-reducer 1..N assignment |

The formula `floor((ordinal - 1) / N) + 1` is identical in every engine.

## 4.6 Reporting metadata

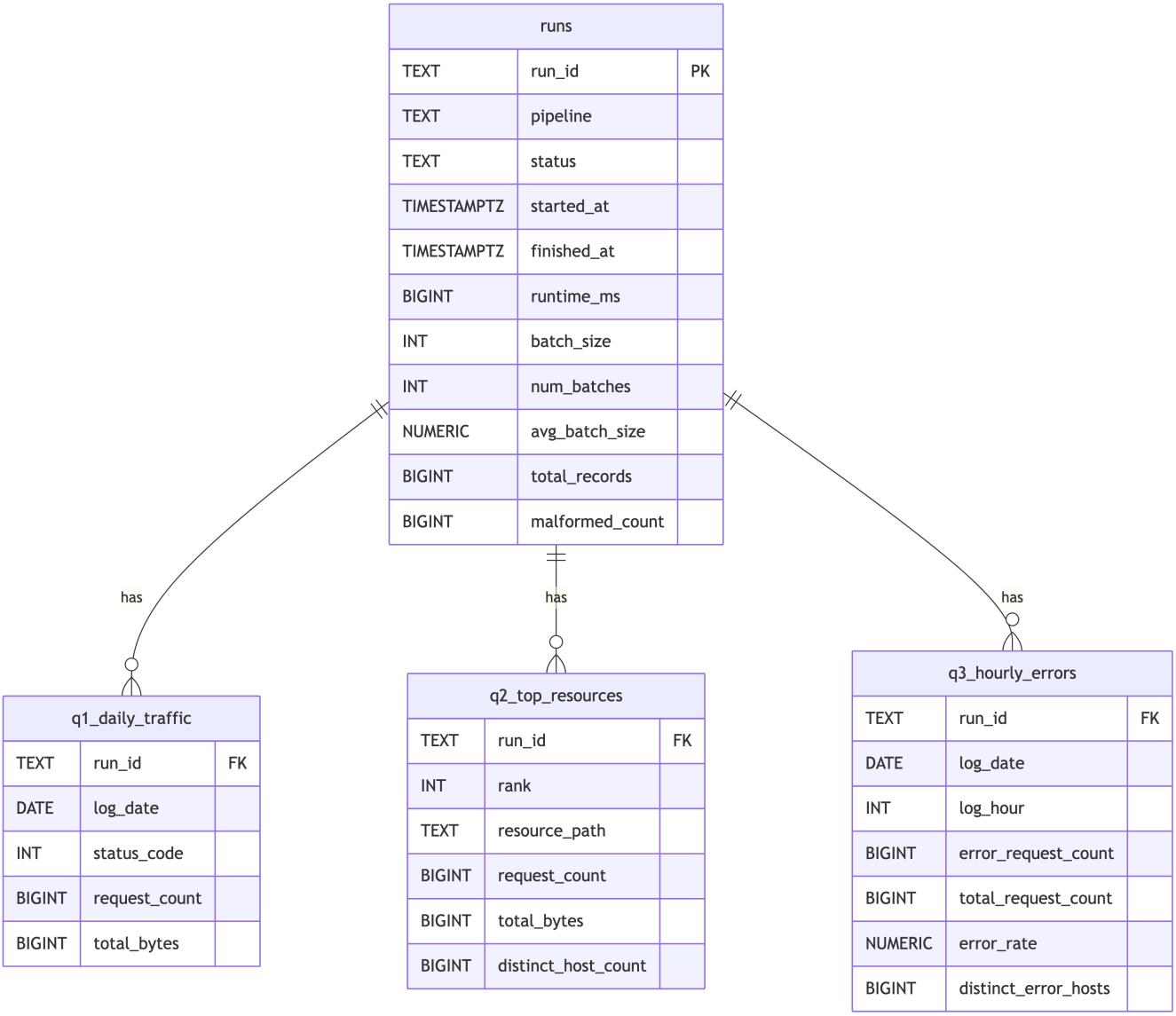
| Field | Value (this run, N = 100,000) |
|---|---|
| `batch_size` | 100,000 |
| `num_batches` | 35 — `max(batch_id)` over `parsed_<run_id>` |
| `avg_batch_size` | 98,902.29 — `total_records / num_batches` per project statement |
| `total_records` | 3,461,580 — parsed (excludes 33 malformed) |
| `malformed_count` | 33 |

---

# 5. Reporting Database Schema

PostgreSQL 16. Four tables: one for run metadata, three for query outputs. Cascaded foreign keys clean up child rows when a `runs` row is deleted.

## 5.1 ER diagram




## 5.2 Schema verbatim

The full DDL is in `sql/schema.sql`. Key points:

- `runs.run_id` is `TEXT` (we use `uuid4().hex`); every query result table references it.
- `runs.status` is `'running'` while a run is in progress and updated to `'completed'` only after the loader writes Q1/Q2/Q3 successfully. If a run crashes mid-flight, the row stays as `'running'` — distinguishable from a finished result.
- `q1_daily_traffic` PK is `(run_id, log_date, status_code)` — guarantees one row per (run, day, status).
- `q2_top_resources` PK is `(run_id, rank)` — exactly 20 rows per run.

- `q3_hourly_errors.error_rate` is `NUMERIC(20, 6)` — locks 6-decimal precision per the parsing spec.
- All FKs include `ON DELETE CASCADE` so dropping a `runs` row cleans up its children.

### 5.3 Project-statement requirements satisfied

The project statement requires the relational store to record **pipeline name, run identifier, batch identifier, time of execution**. Mapping:

| Required | Where in our schema |
|---|---|
| pipeline name | `runs.pipeline` |
| run identifier | `runs.run_id` (referenced by all q* tables) |
| batch identifier | `runs.num_batches` + `runs.batch_size` (the formula recovers `batch_id` for any record) |
| time of execution | `runs.started_at`, `runs.finished_at`, `runs.runtime_ms` |

Reporting metadata required (pipeline, query name, runtime, batch ID, batch size, average batch size) is all materialised in the `runs` row and shown in the report header.

---

## 6. Equivalence Plan Across Pipelines

The project's central requirement is that all four pipelines produce **the same Q1/Q2/Q3 outputs from the same input**. Our framework enforces this through five mechanisms:

1. **One frozen parsing spec.** `PARSING_SPEC.md` is the single source of truth — same regex, same timestamp rule (no UTC conversion), same `bytes='-' → 0` mapping, same malformed definition. Phase 2 backends will reuse the literal regex string.
2. **One batch-id formula.** `floor((ord - 1) / N) + 1` — a single integer expression every engine can implement using its native ordinal primitive (§4.5).
3. **Exact distinct counts.** Every engine must build a `(key, host)` distinct-pair stage and count exactly. No HLL, no `approx_distinct`. This is a frequent source of cross-engine drift; we cut it off at the design level.
4. **A shared `Pipeline` ABC.** `src/pipelines/base.py` defines five lifecycle methods. The orchestrator and loader only use this interface, so any engine that conforms is interchangeable. Phase 2 backends are stubs of this same ABC today.
5. **A common output contract.** Every backend must populate four well-known per-run collections / dirs: `q1_final_<run_id>`, `q2_final_<run_id>`, `q3_final_<run_id>`, plus the `RunStats` returned by `collect_run_stats`. The loader is engine-agnostic — it reads from these four locations regardless of which backend ran.

### 6.1 Phase 2 verification plan

Once Pig / Hive / MR backends are implemented, we will run all four against the **same `--batch-size` and same input**, then diff Q1/Q2/Q3:

```
-- pseudocode for the cross-engine diff (Phase 2)
SELECT log_date, status_code, request_count, total_bytes
```

```sql
FROM q1_daily_traffic WHERE run_id = '<mongo_run>'
EXCEPT
SELECT log_date, status_code, request_count, total_bytes
FROM q1_daily_traffic WHERE run_id = '<hive_run>';
-- expected: zero rows
```

Any non-empty diff exposes a divergence and must be traced back to a parsing-spec violation in the offending engine.

---

# 7. Phase 1 Demonstration

## 7.1 Run command

```bash
psql "postgresql://etl:etl@localhost:5432/etl" -f sql/schema.sql
python -m src.cli run --pipeline mongodb \
    --input ../dataset/ --batch-size 100000
python -m src.cli report --run-id <printed_id>
```

## 7.2 Run metadata (real output)

```
| field           | value                              |
|-----------------|------------------------------------|
| run_id          | 3a5e337e2d6f466590fda039afc86058 |
| pipeline        | mongodb                            |
| status          | completed                          |
| started_at      | 2026-04-28 04:55:30.218668+05:30 |
| finished_at     | 2026-04-28 04:56:59.657279+05:30 |
| runtime_ms      | 89438                              |
| batch_size      | 100000                             |
| num_batches     | 35                                 |
| avg_batch_size  | 98902.29                           |
| total_records   | 3461580                            |
| malformed_count | 33                                 |
```

## 7.3 Q1 sample (295 rows total)

| log_date | status_code | request_count | total_bytes |
|---|---|---|---|
| 1995-07-01 | 200 | 58 033 | 1 617 409 574 |
| 1995-07-01 | 302 | 2 568 | 218 465 |
| 1995-07-01 | 304 | 3 797 | 0 |
| 1995-07-01 | 404 | 316 | 0 |
| 1995-07-03 | 500 | 53 | 0 |
| 1995-08-31 | 200 | … | … |

## 7.4 Q2 — top 20 requested resources

| rank | resource_path | request_count | total_bytes | distinct_host_count |
|---|---|---|---|---|
| 1 | /images/NASA-logosmall.gif | 208 798 | 131 778 402 | 81 518 |
| 2 | /images/KSC-logosmall.gif | 164 976 | 169 364 272 | 85 161 |
| 3 | /images/MOSAIC-logosmall.gif | 127 916 | 40 594 653 | 54 904 |
| 4 | /images/USA-logosmall.gif | 127 082 | 25 993 422 | 54 498 |
| 5 | /images/WORLD-logosmall.gif | 125 933 | 73 796 052 | 54 054 |
| 6 | /images/ksclogo-medium.gif | 121 580 | 629 750 296 | 49 491 |
| 7 | /ksc.html | 83 918 | 563 742 661 | 29 191 |
| 8 | /images/launch-logo.gif | 76 009 | 114 690 489 | 42 558 |
| … | … | … | … | … |
| 20 | /icons/blank.xbm | 28 852 | 13 214 149 | 17 846 |

Top resource is `/images/NASA-logosmall.gif` — consistent with the NASA Kennedy Space Center site's structure (a logo image embedded on every page).

## 7.5 Q3 sample (1,350 rows total)

| log_date | log_hour | error_request_count | total_request_count | error_rate | distinct_error_hosts |
|---|---|---|---|---|---|
| 1995-07-01 | 0 | 24 | 3 565 | 0.006732 | 12 |
| 1995-07-01 | 1 | … | … | … | … |

## 7.6 Test suite

```
$ pytest -q
13 passed in 0.77s
```

- `tests/test_parsing.py` — 4 tests; verifies the master regex against the malformed-rule edge cases.
- `tests/test_batch_id.py` — 5 tests; verifies the batch-id formula across N = 1, 10, 1000, equal-to-total, and partial-last-batch cases.
- `tests/test_aggregations.py` — 4 tests; full ingest → aggregate → final-merge against a 15-line hand-crafted dataset (13 valid + 2 malformed). Asserts Q1 group counts, Q2 tie-break ordering ( `/b` ranks before `/c` when both have `request_count = 3` ), Q3 `error_rate` to 6 decimals, and run-stats invariants ( `num_batches = 2` , `avg_batch_size = 6.5` ).

---

# 8. Status & Phase 2 Roadmap

## 8.1 What's done in Phase 1

- ✅ Frozen parsing spec
- ✅ Postgres schema (4 tables, FKs, cascading delete, indexes, run-status column)
- ✅ `Pipeline` ABC + `RunStats` dataclass
- ✅ MongoDB backend: ingestion, parse-and-tag, per-batch partials, distinct-pair staging, final merge with ranked Q2 and 6-decimal error rate
- ✅ Engine-agnostic loader (Mongo `q*_final` → Postgres)
- ✅ Orchestrator with monotonic timing + transactional runs-row write + cleanup
- ✅ CLI with `run` and `report` subcommands
- ✅ Reporter (tabulate, GitHub flavour)
- ✅ 13-test smoke suite (parsing, batch-id, e2e)
- ✅ End-to-end run on real 3.46M-line dataset; all three result tables populated
- ✅ Code-review pass: premature-commit bug, connection-leak bug, whitespace-only request handling, idempotent ingestion, run-status tracking — all addressed (see §8.3)

## 8.3 Hardening applied after the in-tree code review

| # | Issue | Fix |
|---|---|---|
| Loader committed too early | Loader called `pg_conn.commit()` before the runs-row UPDATE, so a subsequent failure left `runtime_ms = 0` permanently | Removed the commit; orchestrator owns the single-transaction commit |
| Connection leak | `psycopg2.connect()` as a context manager only manages the transaction, not the connection itself | Explicit `pg_conn.close()` in `finally` blocks (orchestrator + reporter) |
| Whitespace-only request line | `"  "` would set `http_method=""` instead of NULL | `req_tokens` now uses `$filter` to drop empty tokens before length check |
| Re-run safety | `insert_many(ordered=True)` aborted the whole batch on a duplicate `_id` retry | `ordered=False` so partial duplicates are reported but ingestion continues |
| Run-status tracking | A failed run left `runtime_ms = 0` indistinguishable from a real result | New `runs.status` column: `'running'` on placeholder INSERT, `'completed'` on UPDATE |
| Empty-input guard | `total_records == 0` would write a meaningless runs row | Orchestrator now raises `RuntimeError` before any DB write |
| Final-collection cleanup | `q*_final` Mongo collections lingered after load | Added to `_STAGING_PREFIXES` |

## 8.2 Phase 2 work

| Item | Owner | Notes |
|---|---|---|
| Apache Pig backend | TBD | |

| Item | Owner | Notes |
|---|---|---|
| | | Same ABC; ordinal via `RANK ... DENSE`; same regex via `REGEX_EXTRACT_ALL`; same distinct-pair pattern via `DISTINCT` then `COUNT_STAR` |
| Apache Hive backend | TBD | Ordinal via `ROW_NUMBER()`; `regexp_extract` for parsing; native `COUNT(DISTINCT ...)` for Q2/Q3 distincts |
| Hadoop MapReduce backend | TBD | Two-pass: first pass assigns ordinals via single reducer; second pass parses and emits per-batch partials; combiners for Q1/Q2/Q3 sums |
| Cross-engine equivalence diff | TBD | SQL `EXCEPT` between any two `run_id`s; Q1/Q2/Q3 expected to be byte-identical |
| Comparative report (Phase 2 PDF) | TBD | Runtime / lines-of-code / difficulty per engine; observations on suitability |
| Video demo | TBD | Live demo of pipeline switching, with all four backends populating the same Postgres tables |

---

## 9. Reproducing the Phase 1 Demo

```
# 1. Mongo (local install via brew, or `docker compose up -d mongo`)
brew services start mongodb-community@7.0

# 2. Postgres (local brew install, or compose-up postgres)
psql -d postgres -c "CREATE ROLE etl LOGIN PASSWORD 'etl';"
psql -d postgres -c "CREATE DATABASE etl OWNER etl;"
psql -U etl -d etl -h localhost -f sql/schema.sql

# 3. Python
python3.11 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"

# 4. Tests
pytest -q

# 5. Real demo
python -m src.cli run --pipeline mongodb \
    --input ../dataset/ --batch-size 100000
python -m src.cli report --run-id <printed_id>
```

The full setup is also documented in `README.md`.